ISM2411 — Lab Week 05

Tiered Discount Calculator — Instructor Facilitation Guide

ISM2411 · Python for Business · USF Muma College of Business

Module 05 · Unit 1 · Foundations

Contents

1 Session Snapshot

Course	ISM2411 — Python for Business
Session	Module 05 Lab — Tiered Discount Calculator
Unit	Unit 1 · Foundations
Class length	75 minutes
Format	Live code-along
Prerequisites	Modules 01–04: variables, types, f-strings, arithmetic/comparison/logical operators
Student-facing lab page	markumreed.github.io/ism2411 — week05_lab
Exercises covered	Exercises 1–6 (required) + Stretch 1/2 (as time allows)
Submission	`discount.py` to Canvas

Every module up to this point produced a boolean (`is_premium`, `product_a_wins`) and then just *printed* it. This is the module where booleans finally start controlling what the program *does* — `if`/`elif`/`else` branching. This is arguably the single most important control-flow concept in the entire course: nearly every remaining module depends on students being fluent with conditionals, and the most common real bug category from here forward is "the wrong branch ran" or "no branch ran when one should have." Budget real time for tracing execution by hand (Exercise 1's reflection question asks for exactly this) — it is the actual skill, more than typing the syntax correctly.

2 Learning Objectives

By the end of this 75-minute session, students should be able to:

1. Write a correct `if`/`elif`/`else` chain and explain why **only one branch ever runs**, even when a later condition would also technically be true.
2. Trace a value through a conditional chain by hand, stating out loud which branch it lands in and why.
3. Combine two conditions with `and` inside a single `if`, and distinguish that from two separate, independent `if` statements.
4. Refactor a nested `if` into a flat one using `and`, and explain why the flat version is easier to read and test.
5. Use `or` and `not` to express "either/or" and "unless" business rules.

3 Before Class — Setup Checklist

☐ Open `discount.py`, empty except for the header comment — build every exercise live.

- ☐ Work through Stretch 2 (the tax-bracket problem) yourself before class — it's the trickiest logic in this lab (marginal, not flat, taxation) and is easy to get subtly wrong live if you haven't rehearsed it once.
- ☐ Decide which values you'll use to test each `if`/`elif`/`else` chain live — this guide uses `total = 250` throughout for consistency with the lab page's own expected output; keep using the same test value across exercises so students aren't recomputing a new mental model each time.

4 Materials Needed

- Instructor laptop + terminal + editor, Python 3.10+
- Students: `discount.py`, same project structure as prior modules

5 Timing Plan (75 minutes)

Time	Segment	Minutes
0:00–0:05	Welcome: "your first real branching logic"	5
0:05–0:15	Exercise 1 — Discount tiers (`if`/`elif`/`else`)	10
0:15–0:21	Exercise 2 — Approval flag	6
0:21–0:29	Exercise 3 — Combine conditions with `and`	8
0:29–0:37	Exercise 4 — Refactor a nested `if`	8
0:37–0:45	Exercise 5 — Anomaly flagging	8
0:45–0:53	Exercise 6 — Boolean logic (`or`, `not`)	8
0:53–1:05	Stretch 1 — Full pricing engine	12
1:05–1:15	Stretch 2 preview + wrap-up, reflection, submission checklist	10

Six required exercises plus Stretch 1 comfortably fill 75 minutes; Stretch 2 (tax brackets) is intentionally positioned as a preview/take-home rather than full live-coded content, since marginal taxation is genuinely tricky to build correctly in real time and deserves unhurried attention if you do walk through it.

6 Segment-by-Segment Walkthrough

6.1 Intro (0:00–0:05)

Say to the class:

> *"Every module so far has computed a `True` or `False` and then just printed it. Today, for the first time, that boolean actually* does *something — it decides which block of code runs. This is `if/elif/else`, and it's the single biggest jump in what your programs can do so far. Watch closely for one specific idea: in a chain of `elif`s, exactly* ***one*** *branch runs, even if a later condition would also be true. That's not obvious the first time you see it, and it's the source of most conditional bugs."*

Do: Open `discount.py`, type the header:

```
# discount.py
# ISM2411 Module 05 Lab – Tiered Discount Calculator
```

6.2 Exercise 1 — Discount Tiers (0:05–0:15, 10 min)

Teaching goal: The core `if/elif/else` pattern, and the crucial "only one branch runs, checked top to bottom" rule.

Say to the class:

> *"A cart total maps to a discount tier: 15% off at $500 or more, 10% at $200 or more, 5% at $100 or more, otherwise nothing. Notice the tiers are checked from* highest to lowest *— that ordering is not an accident, and I want you to predict what breaks if we reverse it."*

Live-code this:

```
# --- Exercise 1 ---
total = float(input("Enter cart total: $"))

if total >= 500:
    discount = 0.15
elif total >= 200:
    discount = 0.10
elif total >= 100:
    discount = 0.05
else:
    discount = 0
```

```
final = total * (1 - discount)
print(f"Discount: {discount*100:.0f}%")
print(f"Final price: ${final:.2f}")
```

Line-by-line explanation:

- `if total >= 500:` — the first condition checked. If a student's cart total is, say, `600`, this is `True`, and Python runs the indented block right below it (`discount = 0.15`) and then **skips every remaining `elif`/`else` in the chain entirely** — it does not go on to check whether `total >= 200` is *also* true (it is, but it's never even evaluated).
- `elif total >= 200:` — "else if": only checked if the `if` above was `False`. This is the line to slow down on: if `total = 250`, the first check (`>= 500`) is `False`, so Python moves to this line, finds `250 >= 200` is `True`, sets `discount = 0.10`, and stops — the `>= 100` check below never runs, even though `250 >= 100` is also technically true.
- `elif total >= 100:` — same pattern, one tier lower.
- `else:` — the catch-all: runs only if every condition above was `False`. No condition needed here — `else` means "everything else."
- `final = total * (1 - discount)` — same discounted-price formula pattern as Module 04, now driven by a `discount` value chosen dynamically instead of hardcoded.
- The indentation itself is not cosmetic — say this explicitly: **the colon and the indented block are what tell Python which lines belong to which branch.** A student whose editor auto-indents inconsistently will hit real, confusing errors; this is worth a 15-second aside now, since indentation-sensitive syntax is new today.

Run it with `total = 250`. Expected output:

```
Discount: 10%
Final price: $225.00
```

Now, live, ask the room to predict the output for `total = 600` and `total = 50` before running each — this rehearsal is exactly what the reflection question asks students to do alone tonight, so do it together now first.

Common student mistakes to watch for:

- Writing four separate `if` statements instead of `if`/`elif`/`elif`/`else` — this is the single most consequential mistake in the whole lab, because it *looks* like it should work but silently doesn't: with `total = 600`, all three of `total >= 500`, `total >= 200`, and `total >= 100` are `True`, so all three `discount = ...` lines would run in sequence, and `discount` ends up holding the *last* one checked (`0.05`), not the intended `0.15`. Deliberately show this live — write four independent `if`s, run it with `total = 600`, and let the room see the wrong (5%, not 15%) output. This single demonstration does more for understanding `elif` than any amount of explanation.
- Ordering the tiers low-to-high instead of high-to-low — with `total = 600` and tiers checked `>= 100` first, the `elif` chain stops at the *first* true condition, which would now be the 5% tier,

never reaching the 15% tier at all. Ask the room to explain why order matters here, given what they just learned about `elif` short-circuiting.

Check for understanding: "For `total = 500` exactly — right at a tier boundary — which discount applies, and why?" (15%, because `>=` includes the boundary value itself; this is worth testing explicitly since boundary conditions are a classic source of off-by-one-style bugs.)

6.3 Exercise 2 — Approval Flag (0:15–0:21, 6 min)

Teaching goal: A second, independent `if`/`else` decision, based on a value computed by the first one — reinforcing that conditionals can chain across multiple decisions, not just within one `elif` block.

Say to the class:

> *"Second, separate decision: does this order need manager approval? Notice this is a brand new `if`/`else`, not another `elif` glued onto the first one — it's checking something completely different."*

Live-code this:

```
# --- Exercise 2 ---
if final > 1000:
    status = "manager_approval_required"
else:
    status = "auto_approved"

print(f"Status: {status}")
```

Line-by-line explanation:

- `if final > 1000:` — this uses `final`, the value *computed* by Exercise 1's block, not user input directly — point out explicitly that this is a fresh, independent `if`/`else`, evaluating a different question about the same underlying transaction.
- Two branches only (`if`/`else`, no `elif`) — since there are only two possible outcomes here, not three or four, there's no need for `elif` at all. This is a good moment to say explicitly: **use exactly as many branches as there are genuinely distinct outcomes** — two outcomes means `if`/`else`, three or more means `elif` chains.

Run it with `final = 637.50` (continuing from Exercise 1's `total = 250` result). **Expected output:**

```
Status: auto_approved
```

Common student mistakes to watch for:

- Writing `elif` here instead of a fresh `if`, mistakenly trying to chain it onto Exercise 1's block — this either causes a `SyntaxError` (if the previous block already had an `else`, since `elif`/`else` cannot follow another `else`) or silently attaches this check to the wrong logical group. Flag explicitly: this is a **new decision about a new question**, not a continuation of the discount-tier decision.

Check for understanding: "What value of `final` is the exact boundary between the two statuses, and which side does it fall on?" (`1000.00` itself is `auto_approved`, since the condition is strictly `>`

`1000`, not >= — a nice contrast with Exercise 1's >= tiers, worth naming as a deliberate difference to read carefully.)

6.4 Exercise 3 — Combine Conditions with `and` (0:21–0:29, 8 min)

Teaching goal: Put two conditions inside a *single* `if` using `and`, and connect this directly back to Module 04's `and` truth-table work — now controlling a branch instead of just producing a printed boolean.

Say to the class:

> *"South-region orders get an extra 2% off, but only on top of an existing tier discount — not as a replacement, not for everyone. Two conditions, joined with `and`, inside one `if`."*

Live-code this (extending Exercise 1's script — `region` is a new input):

```
# --- Exercise 3 ---
region = "South"   # for live testing; normally: input("Region: ")

if region == "South" and total >= 100:
    discount += 0.02
    print("Region bonus applied: additional 2% off")

final = total * (1 - discount)
print(f"Final discount: {discount*100:.0f}%")
print(f"Final price: ${final:.2f}")
```

Line-by-line explanation:

- `if region == "South" and total >= 100:` — both conditions must be `True` for this block to run, exactly the `and` truth table from Module 04, now gating a branch instead of just being printed. Read it explicitly: "is the region South, *and*, is the total at least $100 — both, or neither branch runs."
- `discount += 0.02` — `+=` is shorthand for `discount = discount + 0.02`; this **adds to** whatever `discount` was already set to by Exercise 1's tier logic, rather than replacing it — say this explicitly, since it's the entire point of "on top of," not "instead of."
- Note that this exercise's code runs *after* Exercise 1's tier block has already set `discount` — order matters: this line depends on `discount` already existing with a tier value in it.

Run it with `total = 250`, `region = "South"`. Expected output (continuing from Exercise 1's 10% tier):

```
Region bonus applied: additional 2% off
Final discount: 12%
```

```
Final price: $220.00
```

Common student mistakes to watch for:

- Writing this as two separate `if` statements (`if region == "South":` then a nested `if total >= 100:`) instead of one combined condition — not wrong, exactly (it produces the same result), but this is a deliberate setup for Exercise 4, so if you see it, say "hold that thought" rather than correcting it — Exercise 4 is about to make the *flat-vs-nested* tradeoff the explicit topic.
- Testing only with `region = "South"` and never with a different region — have the room also try `region = "North"` live and confirm the bonus line does *not* print, so the `and` is doing real, visibly-testable work in both directions.

Check for understanding: "If `total = 50` and `region = 'South'`, does the bonus apply?" (No — `total >= 100` is `False`, so `and` makes the whole condition `False` regardless of region; a low-cart South order gets no discount at all under Exercise 1's tiers, so there's nothing to bonus on top of.)

6.5 Exercise 4 — Refactor a Nested `if` (0:29–0:37, 8 min)

Teaching goal: Directly compare a nested `if` chain to the flat, `and`-combined version from Exercise 3 — this is a genuine software-engineering lesson, not just a syntax exercise.

Say to the class:

> *"Here's a nested version of a similar rule — three levels of `if`, each one inside the last. Your job: rewrite it as one flat `if` with `and`, then explain in a comment why the flat version is better."*

Show the nested starting point (from the lab page):

```
# Nested version — rewrite this
if region == "South":
    if total > 100:
        if customer_type == "wholesale":
            extra_discount = 0.03
```

Live-code the refactor:

```
# --- Exercise 4 ---
if region == "South" and total > 100 and customer_type == "wholesale":
    extra_discount = 0.03
    # Flat version reads as one sentence: "South, AND over $100, AND
    # wholesale — all three, or none." The nested version makes you
    # track three separate levels of indentation to find the single
    # line that actually runs, and testing it means constructing test
    # cases for each level separately rather than reasoning about one
    # combined condition at once.
```

Line-by-line explanation:

- Both versions produce **identical behavior** — this is the point to state explicitly first, before discussing why one is "better": refactoring doesn't change what a program does, only how readable/maintainable it is.
- `and` chains left to right, and — just like the two-condition version in Exercise 3 — **all three** conditions must be `True` for the line to run. Three levels of nested `if` and one flat `if` with two `and`s are logically equivalent here specifically because each nested level has exactly one condition and no accompanying `elif`/`else` — flag that this equivalence isn't automatic in every nested-if situation, only in this simple "everything must be true" shape.
- The required comment explaining the "why" — this is the actual graded deliverable, not just the refactored code. A strong comment names **readability** (one line vs. three nested levels) and **testability** (one flat condition can be reasoned about directly, vs. three levels each needing separate consideration).

Common student mistakes to watch for:

- Refactoring correctly but writing a comment that just restates *what* the code does rather than *why* the flat version is better — redirect explicitly: "what does flat code make easier that nested code makes harder?"
- Introducing a subtle behavior change while refactoring — e.g. using `or` instead of `and` by mistake, which would change "all three required" to "any one is enough." If this happens, have the student test both the original nested version and their refactor against the *same* input and compare outputs directly, rather than just re-reading the code.

Check for understanding: "At what point does nesting become genuinely necessary instead of just a style choice — when would flat `and` *not* be a valid substitute for nested `if`s?" (When different levels need *different* `elif`/`else` branches — e.g., if wholesale customers get one treatment and retail customers get a completely different one at the innermost level, that's a real branching structure that flat `and` alone can't express; nesting isn't inherently wrong, it's just usually avoidable when every level shares the same simple "all must be true" shape.)

6.6 Exercise 5 — Anomaly Flagging (0:37–0:45, 8 min)

Teaching goal: A three-way classification using `if`/`elif`/`else` on a *different* kind of business logic (fraud/anomaly detection) than the discount tiers, to confirm the pattern generalizes rather than being tied to pricing specifically.

Say to the class:

> *"Different scenario, same tool. A transaction amount gets classified into one of three categories — this time the interesting boundary is at the* bottom *as well as the top."*

Live-code this:

```
# --- Exercise 5 ---
amount = float(input("Transaction amount: $"))

if amount <= 0:
    print("SUSPICIOUS")
elif amount >= 10000:
    print("LARGE — REVIEW REQUIRED")
else:
    print("NORMAL")
```

Line-by-line explanation:

- `if amount <= 0:` — checked *first*, deliberately, even though it's conceptually the "low" end and "LARGE" is the "high" end — order in an `elif` chain is about which check happens first, not about a value's conceptual position on a number line. Ask the room: "would this still work correctly if I checked the `>= 10000` condition first instead?" (Yes — these two conditions can't both be true for the same `amount`, so order between them doesn't matter here, unlike Exercise 1's tiers, where order was essential. This contrast is worth drawing out explicitly: **order matters in an `elif` chain exactly when a value could satisfy more than one condition** — Exercise 1's tiers overlapped, this exercise's conditions don't.)
- `elif amount >= 10000:` — the upper boundary.
- `else:` — catches everything between, exclusive of both boundaries: `0 < amount < 10000`.

Run it with `amount = -50`, `amount = 500`, `amount = 15000`. Expected output:

```
-50    -> SUSPICIOUS
500    -> NORMAL
15000  -> LARGE — REVIEW REQUIRED
```

Common student mistakes to watch for:

- Using `<` instead of `<=` for the suspicious check, which would let `amount = 0` fall through to `NORMAL` — ask the room to test `amount = 0` specifically and confirm it lands where the spec

(`$0 or negative`) says it should.
- Writing the `>= 10000` check first and the `<= 0` check second — as discussed above, this particular exercise happens to still work either order, but it's worth having a student explain *why* it still works here when it didn't in Exercise 1, to confirm the distinction actually landed.

Check for understanding: "What does this print for `amount = 9999.99`, and what's the largest amount that still prints `NORMAL`?" (`NORMAL` — and the boundary is anything strictly less than `10000`, i.e., `9999.99...` up to but not including `10000` itself, since the LARGE check uses `>=`.)

6.7 Exercise 6 — Boolean Logic: `or` and `not` (0:45–0:53, 8 min)

Teaching goal: Express an "either/or" rule with `or`, and an exclusion rule with `not` — the two operators from Module 04's truth tables that Exercise 3 didn't need yet.

Say to the class:

> *"VIP upgrade: spent more than $2000 this year, OR placed more than 10 orders — either one qualifies, not both required. Then a second rule: excluded if they have any outstanding balance, using `not`."*

Live-code this:

```
# --- Exercise 6 ---
total_spent = 2200
orders_placed = 3
outstanding_balance = 0

vip = (total_spent > 2000 or orders_placed > 10) and not (outstanding_balance > 0)
print(f"VIP upgrade: {vip}")
```

Line-by-line explanation:

- `(total_spent > 2000 or orders_placed > 10)` — `or` returns `True` if **either** side is `True` (or both) — contrast explicitly with Exercise 3/4's `and`, which required **all** sides true. Here, a customer with `total_spent = 2200` and `orders_placed = 3` still qualifies, because the spending condition alone is enough.
- `not (outstanding_balance > 0)` — `not` flips a boolean: if `outstanding_balance > 0` is `True` (they owe money), `not` makes it `False`, and the whole `and` clause becomes `False`, disqualifying an otherwise-qualifying customer. Read the whole line as one sentence: "qualifies on spending or order count, **and** does not have an outstanding balance."
- The parentheses around `(total_spent > 2000 or orders_placed > 10)` are required here for the same order-of-operations reason flagged since Module 04's Stretch B: `and` binds tighter than `or` in Python, so without the parentheses, this would group as `total_spent >`

`2000 or (orders_placed > 10 and not (...))` — a different, wrong rule. This is worth explicitly connecting back to that earlier precedence-trap lesson.

Run it with the values above. Expected output:

```
VIP upgrade: True
```

Now, live, change `outstanding_balance = 50` and re-run — output flips to `False`, even though the spending/orders condition alone is still `True`. This is the entire point of the exercise: "or" grants eligibility, but "not (balance)" can still veto it.

Common student mistakes to watch for:

- Omitting the parentheses around the `or` clause, producing the precedence bug described above — if you see it, have the student test with `total_spent = 2200`, `orders_placed = 3`, `outstanding_balance = 50` specifically; the buggy version incorrectly reports `True` (VIP) because `or`'s left side alone (`total_spent > 2000`) is enough to short-circuit the whole expression to `True` under the wrong grouping, ignoring the balance entirely.
- Writing `outstanding_balance == False` or similar instead of a numeric comparison — `outstanding_balance` is a dollar amount, not a boolean, so the comparison needs to be `> 0`, not an equality check against `True`/`False`.

Check for understanding: "Name a customer (spending, orders, balance) who qualifies on orders alone, not spending, and confirm the balance rule still applies to them the same way." (E.g., `total_spent = 100`, `orders_placed = 15`, `outstanding_balance = 0` → VIP `True`; same customer with `outstanding_balance = 50` → `False`. The point is confirming `not (...)` applies uniformly regardless of *which* side of the `or` qualified them.)

6.8 Stretch 1 — Full Pricing Engine (0:53–1:05, 12 min)

Teaching goal: Combine every rule from Exercises 1–3 into one script that also layers in two new discounts (wholesale, first-order), with a printed breakdown of each step — the closest this lab gets to a realistic, complete business script.

Say to the class:

> *"Everything from today, combined, plus two new rules, applied in a specific order, with a full breakdown printed at the end — like a receipt showing exactly how the final price was reached."*

Live-code this (or walk through it conceptually and hand out the answer key if time is short):

```
# --- Stretch 1 ---
cart_total = 600
region = "South"
customer_type = "wholesale"
first_order = True

breakdown = []

if cart_total >= 500:
    discount = 0.15
elif cart_total >= 200:
    discount = 0.10
elif cart_total >= 100:
    discount = 0.05
else:
    discount = 0
breakdown.append(f"Tier discount: {discount*100:.0f}%")

if region == "South":
    discount += 0.02
    breakdown.append("Regional bonus (South): +2%")

if customer_type == "wholesale":
    discount += 0.05
    breakdown.append("Wholesale extra: +5%")

if first_order:
    discount += 0.03
    breakdown.append("First-order welcome discount: +3%")
```

```
final = cart_total * (1 - discount)
breakdown.append(f"Total discount: {discount*100:.0f}%")
breakdown.append(f"Final price: ${final:.2f}")

for line in breakdown:
    print(line)
```

Line-by-line explanation (highlighting what's new relative to Exercises 1–3):

- `breakdown = []` — an empty list that will collect one string per applied rule, so the final printout shows every step, not just the end result. This is new: earlier exercises just printed as they went; this one accumulates messages and prints them all at the end, which is a small preview of the accumulator pattern coming in Module 06.
- The tier `if`/`elif`/`else` block is identical to Exercise 1, with one addition: `breakdown.append(...)` records which tier applied.
- Three separate `if` blocks follow — South bonus, wholesale extra, first-order discount — **note these are three independent `if`s, not `elif`s**, because a single order can qualify for *all three* simultaneously (unlike Exercise 1's tiers, which are mutually exclusive by construction). This is worth stating explicitly as the key structural difference from Exercise 1: mutually exclusive outcomes need `elif`; independently-stackable bonuses need separate `if`s, exactly like Exercise 3's bonus was separate from Exercise 1's tier.
- `for line in breakdown: print(line)` — a `for` loop over the list, printing each recorded line in order. If `for` loops haven't been covered yet in your section's actual pacing (they're nominally Module 06), this line can be replaced with explicit `print(breakdown[0])`, `print(breakdown[1])`, etc., or simply `print(breakdown)` to show the whole list at once, with a note that a cleaner one-line-per-entry version is coming next module.

Run it with the values shown. Expected output:

```
Tier discount: 15%
Regional bonus (South): +2%
Wholesale extra: +5%
First-order welcome discount: +3%
Total discount: 25%
Final price: $450.00
```

Common student mistakes to watch for:

- Writing the three bonus rules as `elif` instead of independent `if`s — this silently caps the order at receiving only *one* bonus even when it qualifies for several, since an `elif` chain stops at the first true branch. This is the exercise's central "gotcha," directly extending Exercise 1's `elif`-vs-separate-`if` lesson to a case where the *opposite* choice is now correct.
- Applying the bonuses in a different order than specified (tier → regional → wholesale → first-order) — mathematically this particular script produces the same total discount regardless

of order (since += is commutative), but say explicitly that this is a coincidence of *this specific formula* (simple additive stacking), not a general rule — many real discount-stacking rules are order-sensitive (e.g., percentage-of-percentage stacking), and the spec's explicit ordering is worth following as a matter of professional habit even when it doesn't change today's output.

Check for understanding: "If this customer were *not* on their first order, which line disappears from the breakdown, and does the final price change?" (The "First-order welcome discount: +3%" line disappears, `discount` becomes `22%` instead of `25%`, and the final price increases from `$450.00` to `$468.00` — a good quick mental-math check that the room is actually tracking the accumulation, not just pattern-matching the output.)

6.9 Stretch 2 Preview — Tax Brackets (as time allows)

Frame this as a preview/take-home rather than full live-coded content — marginal taxation is genuinely easy to get subtly wrong live, and rewarding to work through carefully rather than rushed:

> *"One more example of tiered logic, from a completely different domain: federal income tax brackets. The trick is that this is **marginal** taxation — each bracket only taxes the income that falls* within *that bracket, not your whole income at that bracket's rate. Someone earning $60,000 does not pay 22% tax on all $60,000 — only the portion above $47,150 is taxed at 22%; the portion from $11,600 to $47,150 is taxed at 12%; the first $11,600 is taxed at 10%."*

If you do walk through it live, the answer key in Appendix A has a verified, working version; the key teaching moment is that a naive "if income is in the 22% bracket, multiply the *whole* income by 22%" approach is wrong and worth explicitly contrasting against the correct marginal calculation.

7 Wrap-Up (last ~10 minutes)

Review the reflection questions out loud:

1. *Trace `order_total = 350` through the discount tier logic, without looking at code.* — A strong answer narrates it as a story: "$350 is not $500 or more, so skip that branch. It IS $200 or more, so `discount = 0.10`, and Python stops checking — it never looks at the `$100` tier even though $350 also qualifies for that." Model this narration style explicitly if students struggle to articulate it.
2. *`if total > 100 and total < 500:` vs. two separate `if` statements* — the key distinction: `and` requires **both** true for the code inside to run at all; two separate `if`s would run **each** block independently whenever its own condition is true, which is a different structure even if, in some specific cases, it happens to produce similar-looking results. Push for a concrete example where they'd actually diverge (e.g., two separate `if`s each printing something — you'd get up to *two* print statements, never possible with the single combined condition, which produces at most one outcome).
3. *Real-world tiered systems (tax brackets, shipping fees, insurance)* — encourage a specific example with actual thresholds if the student can recall one; the point is recognizing `elif`-shaped structure in the wild, which is genuinely everywhere once you start looking.

Review the submission checklist together:

- ☐ File is named `discount.py`
- ☐ Contains Exercises 1–6, each clearly separated (matching the `# --- Exercise N ---` convention from prior modules)
- ☐ `if`/`elif`/`else` used correctly for mutually exclusive tiers; separate `if`s used for independently-stackable bonuses
- ☐ Script runs top to bottom with no errors given valid input

Preview Module 06: "Today's scripts all handled *one* transaction at a time. Next module, you'll process a whole *list* of sales with a `for` loop — and Stretch 1's `breakdown.append(...)` pattern you just saw is a direct preview of the accumulator pattern that's coming."

8 Appendix A — Full Answer Key (`discount.py`)

```python
# discount.py
# ISM2411 Module 05 Lab — Tiered Discount Calculator

# --- Exercise 1 ---
total = float(input("Enter cart total: $"))

if total >= 500:
    discount = 0.15
```

```
elif total >= 200:
    discount = 0.10
elif total >= 100:
    discount = 0.05
else:
    discount = 0

final = total * (1 - discount)
print(f"Discount: {discount*100:.0f}%")
print(f"Final price: ${final:.2f}")

# --- Exercise 2 ---
if final > 1000:
    status = "manager_approval_required"
else:
    status = "auto_approved"
print(f"Status: {status}")

# --- Exercise 3 ---
region = input("Region: ")
if region == "South" and total >= 100:
    discount += 0.02
    print("Region bonus applied: additional 2% off")
final = total * (1 - discount)
print(f"Final discount: {discount*100:.0f}%")
print(f"Final price: ${final:.2f}")

# --- Exercise 4 ---
customer_type = input("Customer type: ")
if region == "South" and total > 100 and customer_type == "wholesale":
    extra_discount = 0.03
    # Flat: one condition to read and test, instead of three
    # nested levels each needing separate consideration.

# --- Exercise 5 ---
amount = float(input("Transaction amount: $"))
if amount <= 0:
    print("SUSPICIOUS")
elif amount >= 10000:
    print("LARGE — REVIEW REQUIRED")
else:
```

```
    print("NORMAL")

# --- Exercise 6 ---
total_spent = float(input("Total spent this year: $"))
orders_placed = int(input("Orders placed: "))
outstanding_balance = float(input("Outstanding balance: $"))
vip = (total_spent > 2000 or orders_placed > 10) and not (outstanding_balance > 0)
print(f"VIP upgrade: {vip}")
```

Stretch 1 (`Full pricing engine`):

```
cart_total = 600
region = "South"
customer_type = "wholesale"
first_order = True

breakdown = []

if cart_total >= 500:
    discount = 0.15
elif cart_total >= 200:
    discount = 0.10
elif cart_total >= 100:
    discount = 0.05
else:
    discount = 0
breakdown.append(f"Tier discount: {discount*100:.0f}%")

if region == "South":
    discount += 0.02
    breakdown.append("Regional bonus (South): +2%")

if customer_type == "wholesale":
    discount += 0.05
    breakdown.append("Wholesale extra: +5%")

if first_order:
    discount += 0.03
    breakdown.append("First-order welcome discount: +3%")

final = cart_total * (1 - discount)
breakdown.append(f"Total discount: {discount*100:.0f}%")
```

```
breakdown.append(f"Final price: ${final:.2f}")

for line in breakdown:
    print(line)
```

Stretch 2 (`Marginal tax brackets`, verified for a single filer, 2024 simplified brackets):

```
income = 60000

tax = 0
remaining = income

if remaining > 47150:
    tax += (min(remaining, 100525) - 47150) * 0.22
    remaining = 47150

if remaining > 11600:
    tax += (remaining - 11600) * 0.12
    remaining = 11600

tax += remaining * 0.10

print(f"Tax owed on ${income:,.2f}: ${tax:,.2f}")
```

Verified results: income `$10,000` → tax `$1,000.00`; income `$30,000` → tax `$3,368.00`; income `$60,000` → tax `$8,253.00`.

9 Appendix B — Extra Practice (only if the class finishes early)

Six required exercises plus Stretch 1 fill the full 75 minutes at a normal pace. If a section moves unusually fast:

Extra — a different tiered system: shipping cost. Orders under $25 pay a flat $6.99 shipping fee; $25–$74.99 pay $3.99; $75 and over ship free. Have students write the `if`/`elif`/`else` independently and test with `order_total = 80`, `order_total = 50`, `order_total = 10`. (`$0.00`, `$3.99`, `$6.99` respectively.)

Extra — one more `and`/`or`/`not` combination. A customer gets free expedited shipping if: they are a VIP (from Exercise 6's logic) AND their order is at least $50, UNLESS the order contains a flagged restricted item. Have students write this as one boolean expression using `and`, `or` (if needed), and `not`, then test it against two or three constructed customer profiles of their own choosing.